

PostgreSQL MCQ

Total points 64/100 ?

The respondent's email (**kriskumarmoradiyaoffice@gmail.com**) was recorded on submission of this form.

0 of 0 points

Name *

Kriskumar Hiteshbhai Moradiya

Exam Questions :

64 of 100 points

✗ Which query retrieves all records from the orders and customers tables where there is a match on customer_id? *0/1

- ☒ SELECT * FROM orders JOIN customers ON orders.customer_id = customers.customer_id; ✗
- ☐ SELECT * FROM orders INNER JOIN customers ON orders.customer_id = customers.customer_id;
- ☐ SELECT * FROM orders JOIN customers WHERE orders.customer_id = customers.customer_id;

Correct answer

- ☒ SELECT * FROM orders INNER JOIN customers ON orders.customer_id = customers.customer_id;

✓ WITH DepartmentCounts AS (
 SELECT department, COUNT(*) AS total_employees
 FROM employees
 GROUP BY department
)
SELECT department, total_employees FROM DepartmentCounts WHERE
total_employees > 5; *1/1

- ☐ Groups by employees instead of departments.
- ☐ Groups employees by department but does not filter.
- ☒ Shows only departments with more than 5 employees. ✓
- ☐ Filters employees before grouping.

✗ Table partitioning in PostgreSQL can be done based on which criteria? * 0/1

- ☒ Indexes ✗
- ☐ Range or List
- ☐ Joins
- ☐ Triggers

Correct answer

- ☒ Range or List

✓ Which query finds employees whose names start with 'A' (case-insensitive)? *1/1

- ☒ SELECT * FROM t_employee WHERE c_name ILIKE 'A%' ✓
- ☐ SELECT * FROM t_employee WHERE c_name SIMILAR TO 'A%'
- ☐ SELECT * FROM t_employee WHERE c_name LIKE 'A%'
- ☐ SELECT * FROM t_employee WHERE c_name = 'A%'

✓ **SELECT COUNT(salary) FROM employee; ***

1/1

- ☐ All rows
- ☐ Only rows with salary = 0
- ☒ Only rows where salary IS NOT NULL ✓
- ☐ Only rows where salary IS NULL

✓ **The PostgreSQL function SETVAL is used for: ***

1/1

- ☐ Setting a column to a default value
- ☐ Creating a new sequence
- ☐ Dropping an existing sequence
- ☒ Setting the current value of a sequence ✓

✓ **Which statement creates a view to show employee name and department?**

*1/1

- ☐ SELECT c_name, c_departmentname INTO emp_dept FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id
- ☐ CREATE VIEW emp_dept (c_name, c_departmentname) FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id
- ☒ CREATE VIEW emp_dept AS SELECT e.c_name, d.c_departmentname FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id ✓
- ☐ CREATE VIEW emp_dept SELECT c_name, c_departmentname FROM t_employee, t_department

✗ Which function returns the length of a string in PostgreSQL? *

0/1

- ☐ LEN()
- ☐ CHAR_LENGTH()
- ☐ SIZEOF()
- ☒ TEXT_SIZE()



Correct answer

- ☒ CHAR_LENGTH()

✓ What is the correct query to list all employees and their corresponding departments, including employees who do not belong to any department? *1/1

- ☒ SELECT [employees.name](#), [departments.name](#) FROM employees LEFT JOIN departments ON employees.department_id = departments.department_id; ✓
- ☐ SELECT [employees.name](#), [departments.name](#) FROM employees RIGHT JOIN departments ON employees.department_id = departments.department_id;
- ☐ SELECT [employees.name](#), [departments.name](#) FROM employees JOIN departments ON employees.department_id = departments.department_id;
- ☐ SELECT [employees.name](#), [departments.name](#) FROM employees LEFT JOIN departments WHERE employees.department_id = departments.department_id;

✓ Which command shows the query execution plan? *

1/1

- ☒ EXPLAIN SELECT * FROM t_employee WHERE c_salary > 50000 ✓
- ☐ PLAN SELECT * FROM t_employee WHERE c_salary > 50000
- ☐ SHOW PLAN FOR SELECT * FROM t_employee WHERE c_salary > 50000
- ☐ ANALYZE PLAN SELECT * FROM t_employee WHERE c_salary > 50000



Which PostgreSQL feature supports **hierarchical queries**? *

1/1



WITH RECURSIVE



NATURAL JOIN



ROLLUP



GROUP BY



Which of the following is used to query a database? *

0/1



TCL (Transaction Control Language)



DML (Data Manipulation Language)



DDL (Data Definition Language)



DCL (Data Control Language)

Correct answer



DML (Data Manipulation Language)



How do you disable a trigger in PostgreSQL? *

0/1



ALTER TABLE table_name DISABLE TRIGGER trigger_name



REMOVE TRIGGER trigger_name



ALTER TRIGGER trigger_name SET DISABLED



DROP TRIGGER trigger_name



Correct answer



ALTER TABLE table_name DISABLE TRIGGER trigger_name



✗ Which of the following is **true about PostgreSQL indexing?** *

0/1

☒ Indexes can only be created on integer columns ✗

☐ Indexes slow down SELECT queries

☐ Indexes are automatically created on primary keys and unique constraints

☐ Indexes cannot be dropped once created

Correct answer

☒ Indexes are automatically created on primary keys and unique constraints

✓ Which query creates a column `full_name` automatically from `first_name` and `last_name`? *1/1

☐ ALTER TABLE t_employee ADD GENERATED full_name TEXT STORED (first_name || last_name)

☒ ALTER TABLE t_employee ADD COLUMN full_name TEXT GENERATED ALWAYS AS (first_name || ' ' || last_name) STORED ✓

☐ ALTER TABLE t_employee ADD COLUMN full_name TEXT AS (first_name || ' ' || last_name)

☐ ALTER TABLE t_employee ADD COLUMN full_name COMPUTED (first_name || last_name)

✓ Which query assigns a row number to employees sorted by salary (highest first)? *1/1

☐ SELECT c_name, COUNT(*) OVER (ORDER BY c_salary DESC) FROM t_employee

☒ SELECT c_name, ROW_NUMBER() OVER (ORDER BY c_salary DESC) FROM t_employee ✓

☐ SELECT c_name, RANK() OVER (ORDER BY c_salary DESC) FROM t_employee

☐ SELECT c_name, DENSE_RANK() OVER (ORDER BY c_salary DESC) FROM t_employee

✓ Which of the following is an example of an inner join in SQL? *

1/1

- ☐ SELECT * FROM table1 FULL OUTER JOIN table2
- ☒ SELECT * FROM table1 INNER JOIN table2 ON [table1.id](#) = [table2.id](#) ✓
- ☐ SELECT * FROM table1 RIGHT JOIN table2
- ☐ SELECT * FROM table1 LEFT JOIN table2

✗ A relation is in 1NF if: *

0/1

- ☒ It has no partial dependencies ✗
- ☐ It has no transitive dependencies
- ☐ It has a single candidate key
- ☐ It has no repeating groups

Correct answer

- ☒ It has no repeating groups

✗ SELECT empid, salary FROM employee e WHERE salary > (SELECT
AVG(salary) FROM employee WHERE deptid = e.deptid);

*0/1

- ☐ Employees with highest salary
- ☒ Employees earning above company average ✗
- ☐ Employees earning above their department average
- ☐ All employees

Correct answer

- ☒ Employees earning above their department average

✓ WITH DesignationSalary AS (

1/1

```
    SELECT
        department,
        designation,
        SUM(salary) AS total_salary
    FROM employees
    GROUP BY department, designation
),
RankedDesignations AS (
    SELECT
        department,
        designation,
        total_salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY
total_salary DESC) AS rank
    FROM DesignationSalary
)
SELECT department, designation, total_salary
FROM RankedDesignations
WHERE rank <= 2;
```

- ☐ It ranks employees individually, rather than by designation.
- ☐ It calculates the sum of salaries within each department.
- ☐ It filters only the top-paying department.
- ☒ It filter first 2 rank to each designation within a department based on total salary (highest first). ✓

✓ Which query extracts the "city" field from a JSON column address? *

1/1

- ☐ SELECT address.city FROM customers
- ☒ SELECT address->>'city' FROM customers ✓
- ☐ SELECT address->'city' FROM customers
- ☐ SELECT json_extract(address, 'city') FROM customers

✓ Which of the following is a **disadvantage of using stored procedures**? * 1/1

- ☒ Tight coupling of business logic with database ✓
- ☐ Easier maintenance of logic inside DB
- ☐ Reduced network traffic
- ☐ Faster execution due to pre-compilation

✗ Which query computes total salary per department, per job role, and overall? *0/1

- ☐ SELECT c_department_id, job_role, SUM(c_salary) FROM t_employee GROUP BY GROUPING SETS ((c_department_id), (job_role), ())
- ☐ SELECT c_department_id, job_role, SUM(c_salary) FROM t_employee GROUP BY ROLLUP(c_department_id, job_role)
- ☐ SELECT SUM(c_salary) FROM t_employee
- ☒ SELECT c_department_id, job_role, SUM(c_salary) FROM t_employee GROUP BY CUBE(c_department_id, job_role) ✗

Correct answer

- ☒ SELECT c_department_id, job_role, SUM(c_salary) FROM t_employee GROUP BY GROUPING SETS ((c_department_id), (job_role), ())

✗ **SELECT *** *
FROM (
 SELECT * FROM employee ORDER BY salary DESC
) e;

0/1

- ☐ Result is always sorted by salary
- ☐ ORDER BY is ignored unless LIMIT is used
- ☒ Causes syntax error
- ☐ Uses index on salary automatically

Correct answer

- ☒ ORDER BY is ignored unless LIMIT is used

✗ **In an Entity-Relationship diagram, what does a diamond shape represent?**

*0/1

- ☐ Entity
- ☐ Primary Key
- ☒ Attribute
- ☐ Relationship

Correct answer

- ☒ Relationship

✓ Which query lists all products and their corresponding orders, including products without orders and orders without products? *1/1

- ☐ SELECT products.product_name, orders.order_id FROM products RIGHT JOIN orders ON products.product_id = orders.product_id;
- ☒ SELECT products.product_name, orders.order_id FROM products FULL OUTER JOIN orders ON products.product_id = orders.product_id; ✓
- ☐ SELECT products.product_name, orders.order_id FROM products LEFT JOIN orders ON products.product_id = orders.product_id;
- ☐ SELECT products.product_name, orders.order_id FROM products JOIN orders ON products.product_id = orders.product_id;

✓ Which practice prevents SQL Injection in PostgreSQL? * 1/1

- ☒ Using parameterized queries with placeholders like \$1, \$2 ✓
- ☐ Using EXECUTE IMMEDIATE with string queries
- ☐ Concatenating user input directly into SQL queries
- ☐ Disabling indexes on sensitive columns

✓ How would you insert a new record into the employees table with emp_id = 101, emp_name = 'John Doe', and hire_date = '2024-01-15'? *1/1

- ☒ INSERT INTO employees (emp_id, emp_name, hire_date) VALUES (101, 'John Doe', '2024-01-15'); ✓
- ☐ INSERT INTO employees SET emp_id = 101, emp_name = 'John Doe', hire_date = '2024-01-15';
- ☐ INSERT INTO employees VALUES (101, 'John Doe', '2024-01-15');
- ☐ INSERT INTO employees (emp_id, emp_name, hire_date) VALUES ('101', 'John Doe', '2024-01-15');

✓ Partitioning in a database is used to: *

1/1

☒ Divide a table into smaller parts ✓

☐ Normalize data

☐ Manage foreign key constraints

☐ Increase redundancy

✗ Which query ensures employees with NULL department_id show as "Unassigned"? *0/1

☐ SELECT c_name, IFNULL(c_department_id, 'Unassigned') FROM t_employee

☐ SELECT c_name, COALESCE(c_department_id, 'Unassigned') FROM t_employee

☐ SELECT c_name, NVL(c_department_id, 'Unassigned') FROM t_employee

☒ SELECT c_name, ISNULL(c_department_id, 'Unassigned') FROM t_employee ✗

Correct answer

☒ SELECT c_name, COALESCE(c_department_id, 'Unassigned') FROM t_employee

✗ Which tool in pgAdmin is used to restore a database from a backup? * 0/1

☐ pg_dump

☐ pg_save

☐ pg_restore

☒ pg_recover ✗

Correct answer

☒ pg_restore

✗ Which PostgreSQL command allows you to **change ownership** of a table? *0/1

- ☒ ALTER OWNER table_name TO new_owner; ✗
- ☐ UPDATE TABLE table_name OWNER new_owner;
- ☐ ALTER TABLE table_name OWNER TO new_owner;
- ☐ GRANT OWNERSHIP TO new_owner;

Correct answer

- ☒ ALTER TABLE table_name OWNER TO new_owner;

✓ Which statement is **TRUE** in PostgreSQL? * 1/1

- ☐ COUNT(1) is faster
- ☐ COUNT() is slower
- ☒ COUNT() and COUNT(1) behave the same ✓
- ☐ COUNT(1) ignores NULL rows

✓ Which query loads data from a CSV file into t_employee? * 1/1

- ☐ IMPORT DATA '/path/employees.csv' INTO t_employee
- ☐ LOAD DATA INFILE '/path/employees.csv' INTO TABLE t_employee
- ☒ COPY t_employee FROM '/path/employees.csv' DELIMITER ',' CSV HEADER ✓
- ☐ BULK INSERT t_employee FROM '/path/employees.csv'

✓ Which constraint ensures that when a department is deleted, its employees are deleted automatically? *1/1

☐ ALTER TABLE t_employee ADD CONSTRAINT fk_dept FOREIGN KEY (c_department_id) REFERENCES t_department(c_department_id) ON DELETE SET NULL

☒ ALTER TABLE t_employee ADD CONSTRAINT fk_dept FOREIGN KEY (c_department_id) REFERENCES t_department(c_department_id) ON DELETE CASCADE ✓

☐ ALTER TABLE t_employee ADD CONSTRAINT fk_dept FOREIGN KEY (c_department_id) REFERENCES t_department(c_department_id) ON UPDATE CASCADE

☐ ALTER TABLE t_employee ADD CONSTRAINT fk_dept FOREIGN KEY (c_department_id) REFERENCES t_department(c_department_id)

✓ SELECT deptid, AVG(salary) * 1/1
FROM employee
WHERE AVG(salary) > 60000
GROUP BY deptid;

☐ Returns correct result

☒ Syntax error ✓

☐ Runtime error

☐ Returns NULL

✗ In PostgreSQL, what is the effect of TRUNCATE on a table? *

0/1

- ☐ Drops the table structure
- ☐ Deletes rows one by one with WHERE clause
- ☒ Removes all rows instantly and cannot be rolled back ✗
- ☐ Removes all rows instantly but can be rolled back if inside a transaction

Correct answer

- ☒ Removes all rows instantly but can be rolled back if inside a transaction

✓ Which query counts how many **different job roles** exist in the employee table? *1/1

- ☐ SELECT COUNT(job_role) FROM t_employee
- ☒ SELECT COUNT(DISTINCT job_role) FROM t_employee ✓
- ☐ SELECT DISTINCT COUNT(job_role) FROM t_employee
- ☐ SELECT job_role FROM t_employee GROUP BY job_role

✗ Which command sets the value of a sequence in PostgreSQL? *

0/1

- ☒ ALTER SEQUENCE seq_name START 100 ✗
- ☐ UPDATE SEQUENCE seq_name TO 100
- ☐ SEQUENCE RESET seq_name 100
- ☐ SETVAL('seq_name', 100)

Correct answer

- ☒ SETVAL('seq_name', 100)

✓ Which of the following is **not true about PostgreSQL CTEs (Common Table Expressions)**? *1/1

- ☐ They provide readability and modularity
- ☐ They can be recursive
- ☒ They always improve query performance ✓
- ☐ They can be used in INSERT, UPDATE, DELETE

✓ How do you select all records from orders where delivery_date is not yet set (i.e., NULL)? *1/1

- ☐ SELECT * FROM orders WHERE delivery_date IS NOT SET;
- ☐ SELECT * FROM orders WHERE delivery_date NOT EXISTS;
- ☒ SELECT * FROM orders WHERE delivery_date IS NULL; ✓
- ☐ SELECT * FROM orders WHERE delivery_date = NULL;

✗ Which command is used by pgAdmin to perform a backup of a PostgreSQL database? *0/1

- ☐ pg_save
- ☒ pg_backup ✗
- ☐ pg_restore
- ☐ pg_dump

Correct answer

- ☒ pg_dump

✗ Which query partitions employee data by department? *

0/1

- ☒ CREATE TABLE emp_partition (c_eid INT, c_name TEXT, c_department_id INT) USING PARTITION(c_department_id) ✗
- ☐ CREATE PARTITION TABLE emp_partition ON t_employee(c_department_id)
- ☐ CREATE TABLE emp_partition PARTITION (c_department_id)
- ☐ CREATE TABLE emp_partition (c_eid INT, c_name TEXT, c_department_id INT) PARTITION BY LIST (c_department_id)

Correct answer

- ☒ CREATE TABLE emp_partition (c_eid INT, c_name TEXT, c_department_id INT) PARTITION BY LIST (c_department_id)

✗ Which transaction property ensures that once a transaction is committed, it will survive system failure?

*0/1

- ☐ Durability
- ☒ Consistency ✗
- ☐ Isolation
- ☐ Atomicity

Correct answer

- ☒ Durability

✓ In DBMS, what is a **prime attribute**? *

1/1

- ☒ An attribute that is part of any candidate key ✓
- ☐ An attribute that always has a unique value
- ☐ An attribute that is used in foreign key reference
- ☐ An attribute that is never part of a key

✓ WITH ProjectStats AS (*1/1
SELECT project_id, COUNT(task_id) AS total_tasks, SUM(hours_spent)
AS total_hours
FROM tasks
WHERE status = 'Completed'
GROUP BY project_id
)
SELECT * FROM ProjectStats
HAVING total_hours > 500;

- ☐ HAVING filters before aggregation, while WHERE filters after.
- ☐ The query will cause an error because WHERE cannot be used with GROUP BY.
- ☒ WHERE filters rows before grouping, while HAVING filters groups after aggregation. ✓
- ☐ Both WHERE and HAVING filter groups before aggregation.

✓ **The data stored in the database at a particular moment is called: *** 1/1

- ☐ Schema
- ☐ Relation
- ☒ Instance ✓
- ☐ Metadata

✓ **SELECT e.emp_id, e.name, d.department_name FROM employees e LEFT *1/1**
JOIN departments d ON e.department_id = d.department_id;

- ☒ The unmatched employees will appear with NULL for department_name. ✓
- ☐ The query will return only matching rows.
- ☐ The query will fail due to unmatched rows.
- ☐ The unmatched employees will be excluded.

✓ One disadvantage of DBMS is: *

1/1

- ☐ It reduces data independence
- ☒ It increases complexity and cost ✓
- ☐ Redundancy is increased
- ☐ It provides better backup mechanisms

✗ Which of the following normal forms eliminates transitive dependencies?

*0/1

- ☐ 2NF
- ☒ 1NF ✗
- ☐ BCNF
- ☐ 3NF

Correct answer

- ☒ 3NF

✓ What will the following query return? *

1/1

```
SELECT region, RANK() OVER (ORDER BY SUM(sales) DESC)
FROM orders
GROUP BY region;
```

- ☐ Total sales for each region only
- ☐ Top selling product of each region
- ☐ Average sales per region
- ☒ Rank of each region based on total sales ✓

✓ In PostgreSQL, what is the difference between **JSON** and **JSONB**? * 1/1

- ☐ They are identical, only naming is different
- ☐ JSONB is stored as plain text, JSON is binary
- ☒ JSONB allows indexing and efficient querying, JSON does not ✓
- ☐ JSON is faster for read operations, JSONB is faster for write

✗ `SELECT department, role, COUNT(emp_id)`
`FROM employees`
`GROUP BY ROLLUP(department, role);` 0/1

- ☐ Generates subtotals for roles within departments and a grand total.
- ☒ Generates all possible groupings (department only, role only, both, and grand total). ✗
- ☐ Groups only by department.
- ☐ Groups only by role

Correct answer

- ☒ Generates subtotals for roles within departments and a grand total.

✓ In DBMS, a **super key** is: * 1/1

- ☐ A key that is always composite
- ☒ Any set of attributes that uniquely identifies a tuple ✓
- ☐ A candidate key used as primary key
- ☐ A minimal super key

✗ Which SQL operation returns only the common rows between two result sets? *0/1

- ☐ FULL JOIN
- ☐ INTERSECT
- ☐ EXCEPT
- ☒ UNION



Correct answer

- ☒ INTERSECT

✓ Temporary tables are automatically dropped when: *

1/1

- ☒ The session ends
- ☐ The user manually drops the table
- ☐ A DELETE statement is executed
- ☐ The table is truncated



✓ Which query finds the maximum salary per department and sorts by department name? *1/1

- ☐ SELECT d.c_departmentname, MAX(e.c_salary) FROM t_employee e, t_department d WHERE e.c_department_id = d.c_department_id ORDER BY d.c_departmentname
- ☒ SELECT d.c_departmentname, MAX(e.c_salary) FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id GROUP BY d.c_departmentname ORDER BY d.c_departmentname
- ☐ SELECT d.c_departmentname, e.c_salary FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id ORDER BY MAX(e.c_salary)
- ☐ SELECT d.c_departmentname, MAX(e.c_salary) FROM t_employee e JOIN t_department d ON e.c_department_id = d.c_department_id



✗ Which of the following queries retrieves the **nth highest salary** in PostgreSQL? *0/1

- ☐ SELECT c_salary FROM t_employee ORDER BY c_salary ASC LIMIT n
- ☒ SELECT DISTINCT c_salary FROM t_employee ORDER BY c_salary ASC LIMIT 1 OFFSET n ✗
- ☐ SELECT MAX(c_salary) FROM t_employee WHERE c_salary < (SELECT MAX(c_salary) FROM t_employee)
- ☐ SELECT DISTINCT c_salary FROM t_employee ORDER BY c_salary DESC LIMIT 1 OFFSET n-1

Correct answer

- ☒ SELECT DISTINCT c_salary FROM t_employee ORDER BY c_salary DESC LIMIT 1 OFFSET n-1

✓ How can you retrieve a list of employees who have the same manager as another employee, using a self-join on the employees table? *1/1

- ☐ SELECT [e1.name](#), [e2.name](#) FROM employees e1 RIGHT JOIN employees e2 ON e1.manager_id = e2.manager_id;
- ☒ SELECT [e1.name](#), [e2.name](#) FROM employees e1 INNER JOIN employees e2 ON e1.manager_id = e2.manager_id WHERE e1.employee_id <> e2.employee_id; ✓
- ☐ SELECT [e1.name](#), [e2.name](#) FROM employees e1 FULL OUTER JOIN employees e2 ON e1.manager_id = e2.manager_id;
- ☐ SELECT [e1.name](#), [e2.name](#) FROM employees e1 LEFT JOIN employees e2 ON e1.manager_id = e2.manager_id;

✗ Which query creates a unique index on employee email? *

0/1

- ☐ CREATE INDEX idx_email UNIQUE ON t_employee(c_email)
- ☒ ALTER TABLE t_employee ADD UNIQUE INDEX idx_email(c_email) ✗
- ☐ CREATE UNIQUE INDEX idx_email ON t_employee(c_email)
- ☐ CREATE INDEX idx_email ON t_employee(c_email)

Correct answer

- ☒ CREATE UNIQUE INDEX idx_email ON t_employee(c_email)

✗ How can you select all departments and their employees, including departments with no employees?

*0/1

- ☐ SELECT departments.department_name, [employees.name](#) FROM departments FULL OUTER JOIN employees ON departments.department_id = employees.department_id;
- ☐ SELECT departments.department_name, [employees.name](#) FROM departments INNER JOIN employees ON departments.department_id = employees.department_id;
- ☐ SELECT departments.department_name, [employees.name](#) FROM departments LEFT JOIN employees ON departments.department_id = employees.department_id;
- ☒ SELECT departments.department_name, [employees.name](#) FROM departments RIGHT JOIN employees ON departments.department_id = employees.department_id; ✗

Correct answer

- ☒ SELECT departments.department_name, [employees.name](#) FROM departments LEFT JOIN employees ON departments.department_id = employees.department_id;

✓ Which query performs a **self join** to find employees who share the same manager? *1/1

- ☐ SELECT e1.c_name, e2.c_name FROM t_employee e1 INNER JOIN t_employee e2 ON e1.c_eid = e2.c_eid
- ☒ SELECT e1.c_name, e2.c_name FROM t_employee e1 JOIN t_employee e2 ON e1.manager_id = e2.manager_id AND e1.c_eid <> e2.c_eid ✓
- ☐ SELECT e1.c_name, e2.c_name FROM t_employee e1 FULL JOIN t_employee e2 ON e1.manager_id = e2.manager_id
- ☐ SELECT e1.c_name, e2.c_name FROM t_employee e1 CROSS JOIN t_employee e2

✓ SELECT deptid, COUNT(*) OVER (PARTITION BY deptid) FROM employee; *

1/1

- ☐ One row per department
- ☐ Error
- ☒ Repeated department count per employee ✓
- ☐ Total employee count

✓ How can you prevent SQL injection attacks in PostgreSQL? *

1/1

- ☐ Using dynamic SQL queries
- ☒ Using parameterized queries ✓
- ☐ Avoiding the use of primary keys
- ☐ Granting all users full access to the database

✓ Which query checks if column details contains a key-value pair { "role": "Manager" }? *1/1

- ☒ SELECT * FROM t_employee WHERE details @> '{"role": "Manager"}' ✓
- ☐ SELECT * FROM t_employee WHERE details -> 'role' = 'Manager'
- ☐ SELECT * FROM t_employee WHERE details #> '{role}' = 'Manager'
- ☐ SELECT * FROM t_employee WHERE details ? 'Manager'

✓ What is the purpose of the ARRAY_AGG function in PostgreSQL? * 1/1

- ☐ Convert an array into individual rows
- ☐ Filter out array elements
- ☐ Sort an array
- ☒ Aggregate values into an array ✓

✗ Which normal form removes partial dependency? * 0/1

- ☐ 3NF
- ☐ 2NF
- ☐ 1NF
- ☒ BCNF ✗

Correct answer

- ☒ 2NF

✓ What will happen if you **execute a stored procedure with OUT parameters** in PostgreSQL? *1/1

- ☐ It will return multiple result sets
- ☒ It will return values via OUT parameters ✓
- ☐ It will raise an error
- ☐ It will not execute

✓ Which query extracts the **year** from a date column hire_date? * 1/1

- ☐ SELECT DATE_PART('YY', hire_date) FROM t_employee
- ☐ SELECT TO_CHAR(hire_date, 'YYYY') FROM t_employee
- ☒ SELECT EXTRACT(YEAR FROM hire_date) FROM t_employee ✓
- ☐ SELECT YEAR(hire_date) FROM t_employee

✓ Which query combines results from sales and returns but excludes duplicate rows? *1/1

- ☐ SELECT * FROM sales UNION ALL SELECT * FROM returns;
- ☒ SELECT * FROM sales UNION SELECT * FROM returns; ✓
- ☐ SELECT * FROM sales COMBINE SELECT * FROM returns;
- ☐ SELECT DISTINCT * FROM sales UNION SELECT DISTINCT * FROM returns;

✓ The ROLLUP operation in SQL is primarily used to: *

1/1

- ☐ Join multiple tables together
- ☐ Exclude specific columns from the result set
- ☒ Aggregate data over a hierarchy of columns ✓
- ☐ Filter data based on specific conditions

✗ How do you update the salary of employees in the sales department by 10%? *0/1

- ☐ UPDATE employees SET salary = salary * 1.10 WHERE department = 'sales';
- ☐ UPDATE employees SET salary = salary + (salary * 0.10) WHERE department = 'sales';
- ☒ UPDATE employees SET salary = salary * 1.10 WHERE department = 'sales' AND salary > 0; ✗
- ☐ UPDATE employees SET salary = salary * 1.10 IF department = 'sales';

Correct answer

- ☒ UPDATE employees SET salary = salary * 1.10 WHERE department = 'sales';

✗ Which of the following is NOT a valid PostgreSQL data type? *

0/1

- ☐ SERIAL
- ☐ BOOLEAN
- ☒ MONEY ✗
- ☐ DATETIMEOFFSET

Correct answer

- ☒ DATETIMEOFFSET

✓ The SUBSTRING function is used to: *

1/1

- ☐ Return the length of a string
- ☐ Replace a substring within a string
- ☒ Extract a portion of a string
- ☐ Concatenate two strings



✗ In PostgreSQL, how can you update the structure of an existing view? *

0/1

- ☐ MODIFY VIEW view_name
- ☐ ALTER VIEW view_name
- ☒ UPDATE VIEW view_name
- ☐ RENAME VIEW view_name



Correct answer

- ☒ ALTER VIEW view_name

✓ A BEFORE INSERT trigger in PostgreSQL executes: *

1/1

- ☒ Before a new row is inserted into the table
- ☐ After a row is updated in the table
- ☐ After a new row is inserted into the table
- ☐ Before a row is deleted from the table



✗ Which query returns all employees whose JSON column details contains key "skills"? *0/1

- ☒ SELECT * FROM t_employee WHERE details @> '{"skills":[]}' ✗
- ☐ SELECT * FROM t_employee WHERE details LIKE '%skills%'
- ☐ SELECT * FROM t_employee WHERE details->'skills' IS NOT NULL
- ☐ SELECT * FROM t_employee WHERE details ? 'skills'

Correct answer

- ☒ SELECT * FROM t_employee WHERE details ? 'skills'

✓ What does the ROW_NUMBER() function do in PostgreSQL? * 1/1

- ☐ Assigns numbers in descending order by default.
- ☐ Skips ranking numbers for duplicate values.
- ☐ Assigns the same rank to duplicate values.
- ☒ Assigns a unique sequential number to each row within a partition. ✓

✗ In PostgreSQL, which function is used to get current date and time with time zone? *0/1

- ☐ CURRENT_TIME
- ☐ NOW()
- ☒ LOCALTIMESTAMP ✗
- ☐ CURRENT_DATE

Correct answer

- ☒ NOW()

✓ Which query selects product_id, product_name, and price from the products table for products with a price greater than 100, sorted by price in descending order? *1/1

- ☐ SELECT product_id, product_name, price FROM products ORDER BY price DESC WHERE price > 100;
- ☒ SELECT product_id, product_name, price FROM products WHERE price > 100 ORDER BY price DESC; ✓
- ☐ SELECT product_id, product_name, price FROM products WHERE price > 100 SORT BY price DESC;
- ☐ SELECT product_id, product_name, price FROM products ORDER BY price > 100 DESC;

✓ Which query correctly defines a CTE to select customers with id greater than 10? *1/1

- ☐ WITH cte_customers AS SELECT id FROM customers WHERE id > 10;
- ☐ SELECT * FROM WITH cte_customers AS (SELECT * FROM customers WHERE id > 10);
- ☐ WITH cte_customers (SELECT * FROM customers WHERE id > 10) RETURN *;
- ☒ WITH cte_customers AS (SELECT * FROM customers WHERE id > 10) SELECT * FROM cte_customers; ✓

✓ Which join returns all employees with their departments, even if some employees are not assigned to any department? *1/1

- ☐ SELECT * FROM t_employee CROSS JOIN t_department
- ☒ SELECT * FROM t_employee LEFT JOIN t_department ON t_employee.c_department_id = t_department.c_department_id ✓
- ☐ SELECT * FROM t_employee FULL JOIN t_department ON t_employee.c_department_id = t_department.c_department_id
- ☐ SELECT * FROM t_employee INNER JOIN t_department ON t_employee.c_department_id = t_department.c_department_id

✓ Which of the following normal forms have atomic values? * 1/1

- ☐ 3NF
- ☐ 2NF
- ☐ BCNF
- ☒ 1NF ✓

✓ SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department HAVING SUM(salary) > 50000; *

- ☐ Filters rows before grouping.
- ☐ Filters individual employee salaries greater than 50,000.
- ☐ Displays all departments.
- ☒ Filters groups where the total salary exceeds 50,000. ✓

✓ Which SQL statement is used to create a stored procedure in PostgreSQL?

*1/1

- ☐ CREATE TRIGGER
- ☐ CREATE QUERY
- ☐ CREATE FUNCTION
- ☒ CREATE PROCEDURE



✓ SELECT emp_id, department,
ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary
DESC) AS row_num
FROM employees;

*1/1

- ☐ Orders the entire table by department and salary but does not assign row numbers.
- ☐ Assigns the same row number to employees with the same salary.
- ☐ Assigns a sequential row number to all employees in the table.
- ☒ Assigns a sequential row number within each department based on salary in descending order.



✓ Which of the following queries groups the customers by name and returns the count for each group?

*1/1

- ☒ SELECT name, COUNT(*) FROM customers GROUP BY name;
- ☐ SELECT name, COUNT(*) FROM customers SORT BY name;
- ☐ SELECT name, COUNT(*) FROM customers ORDER BY name;
- ☐ SELECT name, SUM(*) FROM customers GROUP BY name;



✓ How does RANK() differ from ROW_NUMBER() in PostgreSQL? * 1/1

- ☐ Both functions work identically.
- ☐ RANK() assigns numbers in reverse order.
- ☒ RANK() skips numbers for duplicate values, while ROW_NUMBER() does not. ✓
- ☐ ROW_NUMBER() assigns the same rank to duplicate values, while RANK() does not.

✗ A set of attributes that uniquely identifies each record but may contain extra attributes is called? *0/1

- ☒ Candidate Key ✗
- ☐ Primary Key
- ☐ Foreign Key
- ☐ Super Key

Correct answer

- ☒ Super Key

✓ SELECT DISTINCT deptid FROM employee; * 1/1

- ☐ SELECT UNIQUE deptid FROM employee;
- ☐ SELECT deptid FROM employee ORDER BY deptid;
- ☒ SELECT deptid FROM employee GROUP BY deptid; ✓
- ☐ SELECT deptid, COUNT(*) FROM employee;

✓ Which SQL clause is missing in the following query to filter employees whose salary is greater than 50,000 before grouping the records? *1/1

```
SELECT deptid, COUNT(*) FROM employee GROUP BY deptid HAVING  
COUNT(*) > 3  
ORDER BY deptid;
```

☐ SELECT

☐ GROUP BY

☒ WHERE ✓

☐ HAVING

✓ What is the purpose of the GROUPING SETS clause in SQL? * 1/1

☐ To limit the number of rows in the result

☒ To perform grouping on multiple sets of columns ✓

☐ To rename columns in a group

☐ To apply filters to grouped data

✗ Which command is used to load bulk data into a PostgreSQL table? * 0/1

☐ BULK INSERT

☒ LOAD DATA ✗

☐ IMPORT

☐ COPY

Correct answer

☒ COPY

✗ **WITH SalesAnalysis AS (** * 0/1
 SELECT department, role, SUM(sales) AS total_sales
 FROM employees
 GROUP BY CUBE(department, role)
)
 SELECT * FROM SalesAnalysis;

- ☐ Generates all possible groupings (department, role, both, and grand total).
- ☐ Groups only by department.
- ☒ Generates all possible groupings (department, role, and grand total). ✗
- ☐ Groups only by role

Correct answer

- ☒ Generates all possible groupings (department, role, both, and grand total).

✓ **Which of the following is an advantage of using a DBMS?** * 1/1

- ☒ Data inconsistency is reduced ✓
- ☐ Redundancy is increased
- ☐ It is harder to enforce integrity constraints
- ☐ Unauthorized access to data is easier

✓ **SELECT empid,** * 1/1
 salary,
 RANK() OVER (PARTITION BY deptid ORDER BY salary DESC)
 FROM employee;

- ☐ Ranks employees globally
- ☒ Ranks employees department-wise ✓
- ☐ Removes duplicate salaries
- ☐ Aggregates salaries

✓ CREATE INDEX idx_emp_name ON employee(empname); *

1/1

Which query will **use the index**?

- ☒ SELECT * FROM employee WHERE empname = 'Ketan'; ✓
- ☐ SELECT * FROM employee WHERE LOWER(empname) = 'ketan';
- ☐ SELECT * FROM employee WHERE empname LIKE '%ket%';
- ☐ SELECT * FROM employee WHERE empname::text = 'Ketan';

✓ In PostgreSQL, what will happen if you **drop a view**? *

1/1

- ☐ Database is dropped
- ☒ The underlying base tables remain unaffected ✓
- ☐ All dependent tables are deleted
- ☐ The underlying base tables are deleted

✗ What does SETVAL function do in PostgreSQL? *

0/1

- ☐ Resets sequence counter to a specified value
- ☒ Sets a new default value for a column ✗
- ☐ Sets primary key for a table
- ☐ Sets transaction isolation level

Correct answer

- ☒ Resets sequence counter to a specified value

✗ CREATE INDEX idx_active_emp ON employee(empid) *
WHERE status = 'ACTIVE';

0/1

Which query uses this index?

- ☐ SELECT * FROM employee WHERE status = 'ACTIVE';
- ☐ SELECT * FROM employee WHERE status = 'INACTIVE';
- ☒ SELECT * FROM employee;
- ☐ SELECT * FROM employee WHERE empid > 10;

✗

Correct answer

- ☒ SELECT * FROM employee WHERE status = 'ACTIVE';

This form was created inside of CLARENT INSTITUTE PRIVATE LIMITED. - [Contact form owner](#)

Does this form look suspicious? [Report](#)

Google Forms

